KARE

KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

(DEEMED TO BE UNIVERSITY)
Anand Nagar, Krishnankoil, Tamil Nadu

PROJECT DOCUMENTATION

FINANCETRACK

Personal Finance Management System

A MERN Stack Web Application

Submitted by

S. No.	Student Name	Register Number
1	K.B. Charan	99230040358
2	V. Shiva Kumar	99230041082
3	A. Sushma	99230040251
4	V. Padma	99230040802
5	B. Harsha	99230040080

Project Guide: _____
Academic Year: _____

TABLE OF CONTENTS

1. Abstract
2. 1. Introduction
3. 2. Problem Statement
4. 3. Objectives
5. 4. Proposed System And Scope
6. 5. Technology Stack
7. 6. System Architecture
8. 7. Module Description
9. 8. Authentication And Authorization
10. 9. Automated Budget Alert Workflow
11. 10. Functional And Non-Functional Requirements
12. 11. Testing And Validation
13. 12. Deployment
14. 13. Conclusion
15. 14. Future Scope
16. References

ABSTRACT

FinanceTrack is a MERN stack web application designed to help individuals record, organize, monitor and understand their personal finances in one place. The system brings income and expense tracking, category-based monthly budgets, dashboard summaries, reports and administrative controls into a single website. Users can create and manage transactions, review balances and spending patterns, and export filtered transaction data as CSV. Authentication is implemented using JSON Web Tokens (JWT), while passwords are hashed using bcryptjs. A budget monitoring service checks spending after transaction changes and sends email alerts through the Brevo API when a category reaches 90% or 100% of its monthly budget. MongoDB Atlas stores application data, and Recharts presents financial summaries visually.

1. INTRODUCTION

Managing personal finances requires users to keep track of income, expenses, budgets and spending patterns. When this information is stored in notebooks or separate spreadsheets, it can become difficult to maintain accurate records, understand monthly spending and identify budget overruns. Preparing summaries and comparing expenses also takes additional time.

FinanceTrack addresses these challenges through a centralized personal finance management website. It allows users to record income and expenses, organize transactions by category and date, set monthly category budgets, view dashboard summaries and generate reports. The application also provides administrative functions for managing users and system categories.

2. PROBLEM STATEMENT

Personal financial records are often scattered across notebooks and separate files. This manual approach provides limited visibility into spending, makes it difficult to monitor category-wise limits and increases the effort required to prepare monthly reports. Without timely monitoring, users may exceed their planned budgets without noticing.

The project therefore focuses on providing a unified web-based system for recording financial activity, monitoring monthly budgets and presenting understandable summaries.

3. OBJECTIVES

• Provide a single place to record income and expenses.

• Organize transactions using categories and dates.

• Allow users to create and manage monthly budgets for expense categories.

• Display income, expenses, balance, trends and category summaries on a dashboard.

• Support transaction search, filtering and CSV report export.

• Secure user accounts using password hashing and JWT-based authentication.

• Provide role-based access for USER and ADMIN accounts.

• Send automated email notifications when category spending reaches defined budget thresholds.

• Allow administrators to manage users, categories and application statistics.

4. PROPOSED SYSTEM AND SCOPE

FinanceTrack combines everyday personal finance tasks in one website. A registered user can manage their own financial records, review dashboard information, maintain budgets and access reports. The system separates financial activity from account and administration tasks. Administrators can manage users, system categories and

application statistics.

The project scope includes the landing page, registration and login, dashboard, transactions, budgets and budget analysis, reports, profile, change password, admin dashboard, user management and category management. The application is deployed with the frontend on Vercel and the backend on Render.

5. TECHNOLOGY STACK

Frontend: React 18, Vite, React Router, Axios and Recharts.

Backend: Node.js 18, Express.js, JSON Web Tokens (JWT) and bcryptjs.

Database: MongoDB Atlas and Mongoose.

Supporting tools/services: Brevo API, React Hot Toast and date-fns.

Deployment: Vercel for the frontend and Render for the backend.

6. SYSTEM ARCHITECTURE

The application follows a client-server architecture. The React frontend contains pages, forms and charts. It communicates with the Node.js and Express backend through HTTP requests carrying JSON data. Express routes direct requests to controllers and apply authentication or role-checking middleware where required. Mongoose provides document validation and database operations. MongoDB Atlas stores users, transactions, categories, monthly budgets and alert records.

The main request flow is: React frontend → Express API → Mongoose → MongoDB Atlas → JSON response to React. The backend also invokes the budget-check service after transaction changes. When a budget threshold is reached, the service uses the Brevo Email API to send the appropriate notification.

7. MODULE DESCRIPTION

7.1 Landing, Registration and Login

The landing page introduces the application. Users can register with their name, email and password, and then log in to access protected features.

7.2 Dashboard

The dashboard presents income, expenses, balance and visual summaries. Recharts is used to display financial information in chart form.

7.3 Transaction Management

Users can add, view, edit, delete, search and filter income and expense records. Transactions are organized by category and date.

7.4 Budget Management and Analysis

Users can create and manage monthly spending limits for expense categories. Budget analysis helps users compare category spending with the planned limit.

7.5 Reports

Users can apply date and category filters, review transaction records and export reports in CSV format.

7.6 Categories

The application supports system categories and personal categories. Administrators can manage system categories.

FINANCETRACK - DASHBOARD SCREENSHOT

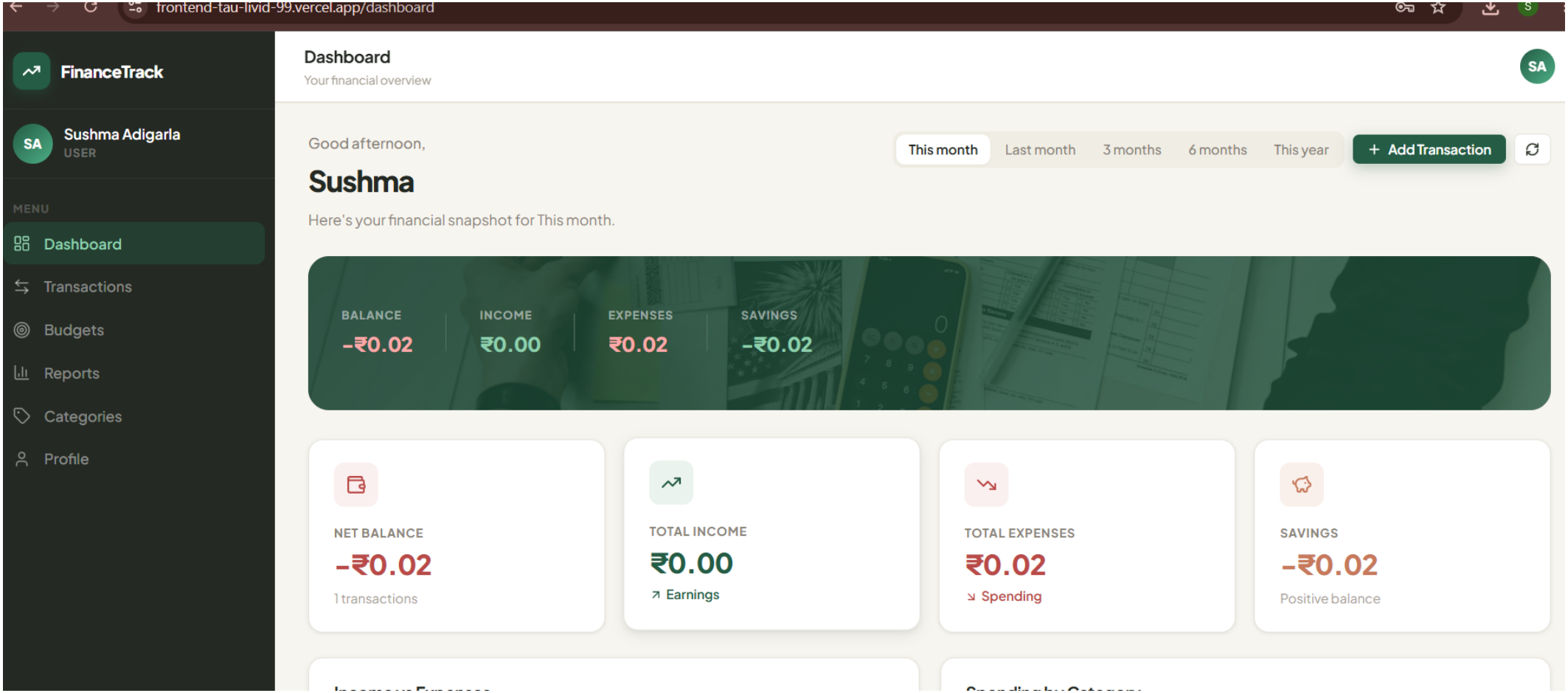


Figure 1. Dashboard — income, expenses, balance and savings overview

FINANCETRACK - TRANSACTION MANAGEMENT SCREENSHOT

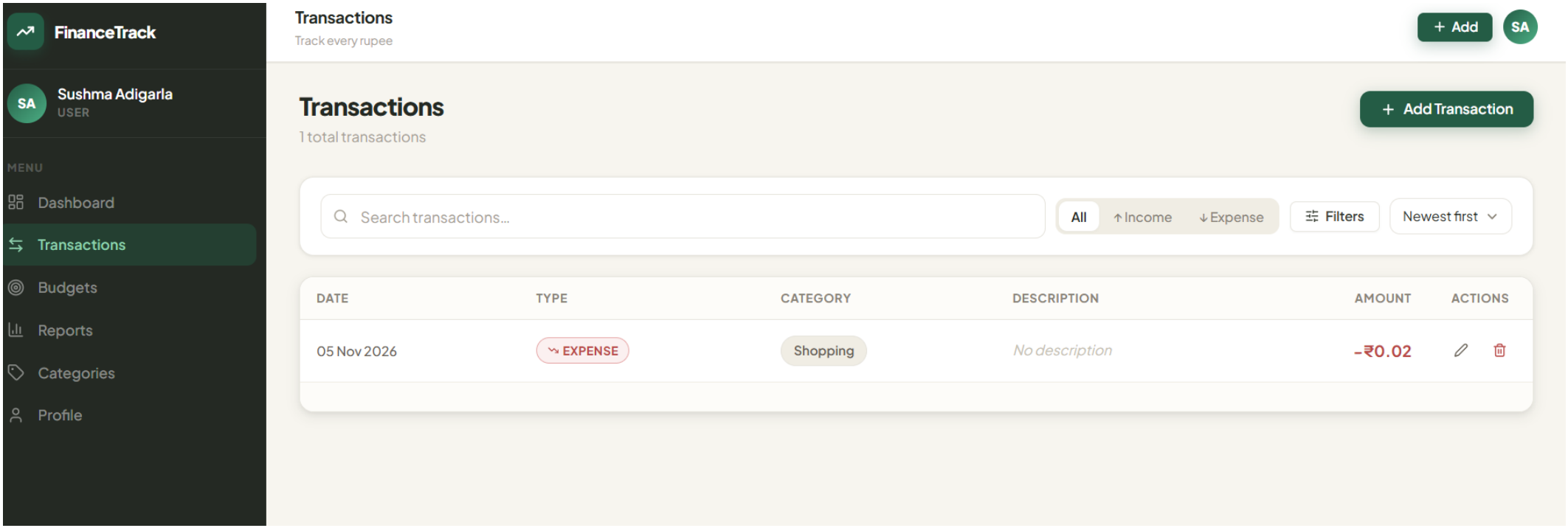


Figure 2. Transaction Management — transaction list, search and filters

FINANCETRACK - BUDGET MANAGEMENT SCREENSHOT

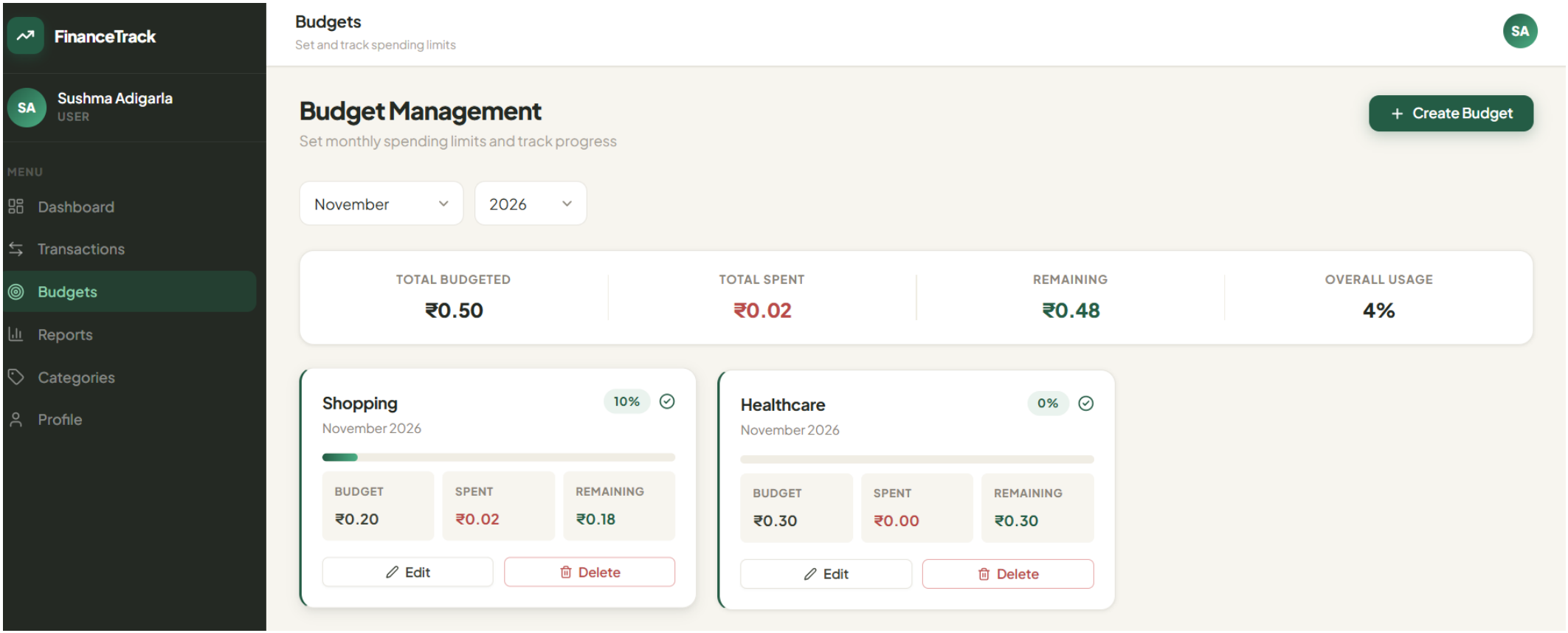


Figure 3. Budget Management — monthly limits and spending progress

FINANCETRACK - CATEGORY MANAGEMENT SCREENSHOT

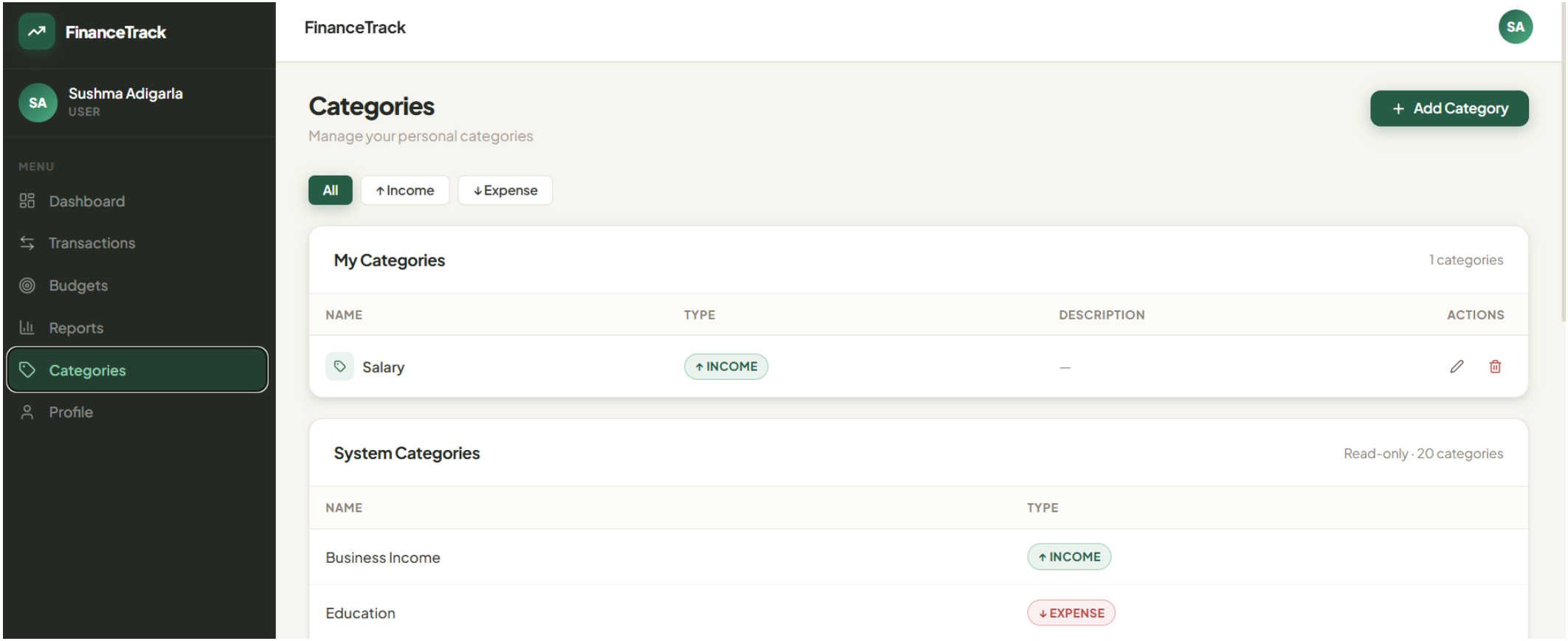


Figure 4. Category Management — personal and system categories

7.7 Profile and Change Password

Users can view and manage account profile information and change their password.

7.8 Admin Dashboard and User Management

Administrators can view application statistics and manage users. Admin-only operations are restricted through role-based authorization.

8. AUTHENTICATION AND AUTHORIZATION

Authentication verifies the identity of a user, while authorization determines which resources the authenticated user is permitted to access. During registration, the password is hashed using bcryptjs. During login, the supplied password is verified and a JWT is issued. Axios sends the token with requests to protected APIs.

The application supports USER and ADMIN roles. The protect middleware verifies the JWT before protected routes are accessed. The requireAdmin middleware restricts administrator-only APIs. User records are scoped to the signed-in account. A 401 response indicates an unauthenticated request; a 403 response indicates that the user is authenticated but not authorized for the requested operation.

9. AUTOMATED BUDGET ALERT WORKFLOW

The budget alert workflow is triggered when an expense is added, updated or deleted. The backend recalculates spending for the affected category and month, then compares the total with the configured monthly budget.

• At 90% of the budget: a warning email is sent through Brevo.

• At 100% or above: a budget-limit-reached email is sent.

• Alert status is stored in MongoDB.

Saving alert records helps prevent the same threshold notification from being sent repeatedly. This workflow gives users timely information about their category-wise spending.

10. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements:

• Register, log in and manage a user profile.

• Create, view, edit and delete transactions.

• Create and manage monthly category budgets.

• Filter transaction records and view reports.

• Authenticate protected API requests.

• Validate user input and handle errors.

• Calculate dashboard and budget summaries.

• Send budget-threshold email notifications.

• Allow administrators to manage users and categories.

Non-functional expectations:

• Security: password hashing, JWT authentication and role-based access.

• Usability: responsive pages, clear feedback and visual summaries.

• Maintainability: modular routes, controllers, models and reusable components.

11. TESTING AND VALIDATION

The PPT describes the system features and workflows but does not provide measured test-case results or performance figures. The following functional checks are appropriate for validating the implemented application:

• Registration and login: verify valid account creation, password hashing and successful login.

• Protected access: verify that requests without a valid JWT are rejected.

• Role access: verify that USER accounts cannot access ADMIN-only APIs.

• Transaction operations: verify add, view, edit, delete, search and filter operations.

• Budget calculation: verify category spending against the monthly budget after transaction changes.

• Alert thresholds: verify warning notification at 90% and limit-reached notification at 100% or above.

• Reports: verify date/category filtering and CSV export.

• Data persistence: verify that users, transactions, categories, budgets and alert records are stored and retrieved correctly.

Actual pass/fail outcomes should be entered after executing these checks in the deployed application.

12. DEPLOYMENT

The frontend is deployed on Vercel and the backend is deployed on Render. The application uses MongoDB Atlas as its cloud database. The live website URL supplied in the presentation is:

https://frontend-tau-livid-99.vercel.app/

The frontend communicates with the backend through the configured API endpoints. Brevo is used by the backend for budget alert email notifications.

13. CONCLUSION

FinanceTrack provides a centralized platform for personal finance management by combining income and expense tracking, monthly category budgets, dashboard summaries, reports and administrative features. The React frontend communicates with Express REST APIs, while MongoDB Atlas stores the application data. JWT-based authentication and role-based access help protect user and administrator functions. Automated budget alerts through Brevo provide notifications when spending approaches or exceeds a category limit. Overall, the application aims to make it easier for users to track, plan and understand their money.

14. FUTURE SCOPE

The project presentation identifies future scope but does not list specific planned enhancements. Possible extensions consistent with the current system include adding recurring transaction support, more customizable reports, additional dashboard comparisons, enhanced notification preferences and mobile-friendly improvements. These are proposed enhancements and are not represented as existing features.

REFERENCES

1. FinanceTrack project presentation, “MERN Stack Web Application – FinanceTrack: Personal Finance Management System” (provided project slides).

2. React documentation: https://react.dev/

3. Node.js documentation: https://nodejs.org/docs/

4. Express.js documentation: https://expressjs.com/

5. MongoDB documentation: https://www.mongodb.com/docs/

6. Mongoose documentation: https://mongoosejs.com/docs/

7. Vite documentation: https://vite.dev/guide/

8. Recharts documentation: https://recharts.org/

9. Brevo API documentation: https://developers.brevo.com/